

GridWise — 3-Minute Video Script

Total Target Duration: 2:50 – 3:00

Speaker: Team member (screen recording + voiceover)

Screen: Code editor / terminal / architecture diagram / live API demo

SECTION 1 — Problem & Motivation (0:00 – 0:35)

[SCREEN: Show the problem statement title page or a slide with the challenge name]

VOICEOVER:

"Hi, this is our solution for the BUP CSE Fest 2026 Hackathon — **GridWise**, an LLM-Assisted Smart Campus Energy Optimization Service.

The challenge is this: a university campus runs on three energy sources — **grid electricity**, **rooftop solar panels**, and a **battery storage system**. Each hour has different demand, solar availability, and grid tariffs. Campus operators also send **natural-language notes** — like 'solar panels are being cleaned from noon to 2 PM' or 'keep at least 120 kWh in the battery during evening peak.'

Our job is to **understand** those free-text notes using an LLM, **validate** them with deterministic guardrails, **apply** them as mathematical constraints, and **optimize** a 24-hour battery schedule that **minimizes total grid electricity cost** — while respecting every physical and operational rule."

SECTION 2 — Architecture Overview (0:35 – 1:30)

[SCREEN: Show the 4-stage pipeline diagram from the README, or draw it on a whiteboard/slide]

[Energy Data + Operator Notes]



Stage 1: LLM Interpreter
(Google Gemini 2.0 Flash)



Stage 2: Deterministic
Guardrail Validator



Stage 3: PuLP MILP Optimizer
(CBC Solver)



Stage 4: Schedule Replayer
& Metric Verifier

VOICEOVER:

"Our system is a strictly **sequential 4-stage pipeline** — and this separation is the core architectural decision.

Stage 1 — LLM Interpreter. We use **Google Gemini 2.0 Flash** with structured JSON output mode and temperature zero. The LLM receives operator notes along with battery context and classifies each note into one of six canonical directive types — `solar_reduction`, `minimum_battery_reserve`, `no_charge_window`, `no_discharge_window`, `max_grid_window`, or `no_op` for irrelevant notes. The system prompt includes extensive time-window examples to enforce the start-inclusive, end-exclusive hour convention — so '1 PM to 3 PM' always maps to hours 13 and 14, never 15.

Stage 2 — Deterministic Guardrails. This is pure Python with **zero hallucination risk**. We treat the LLM output as *untrusted data*. The guardrail enforces: only allowed directive types, hours must be unique integers 0–23 in ascending order, solar factor must be between 0 and 1, reserve values can't exceed battery capacity, and grid caps must be non-negative. If anything is malformed, we safely coerce it to `no_op` rather than crashing or silently inventing a constraint.

Stage 3 — PuLP MILP Optimizer. We formulate a mixed-integer linear program using the PuLP library with the CBC solver. The objective function minimizes total grid cost. Constraints include: hourly energy balance, solar usage limits after applying any `solar_reduction`, battery state-of-charge transitions, charge and discharge rate limits, mutual exclusivity of charging and discharging via binary indicator variables, and critically — **end-of-day battery neutrality**, so the battery can't be drained as a free energy source. All operator directives become hard constraints in the LP.

Stage 4 — Replayer & Verifier. After the solver produces an optimal schedule, we **independently replay** every hour to verify energy balance, battery bounds, directive compliance, and end-of-day neutrality. We also recalculate `total_grid_kwh`, `total_cost_bdt`, and `peak_grid_kwh` directly from the hourly plan so reported totals always match."

SECTION 3 — Key Implementation Choices (1:30 – 2:10)

[SCREEN: Briefly show relevant code files — `llm_interpreter.py`, `guardrails.py`, `optimizer.py`]

VOICEOVER:

"A few key design decisions worth highlighting:

First — Structured JSON generation. We set Gemini's `response_mime_type` to `application/json` with temperature zero, so the LLM returns clean JSON arrays directly — no regex extraction needed.

Second — Deterministic fallback. If the Gemini API key is missing or the API is unreachable, the service automatically switches to a built-in heuristic parser. This means local testing never crashes, and we have a safety net during judging.

Third — The guardrail layer is the trust boundary. The LLM is powerful but probabilistic. The guardrails are rigid and deterministic. By separating these strictly, we guarantee that no hallucinated directive ever reaches the optimizer.

Fourth — Technology stack. We chose **FastAPI** for the web framework — it gives us automatic request validation via Pydantic v2, OpenAPI docs, and async support. **PuLP with CBC** for the

solver — it's a proven LP/MILP framework. And **Google Gemini 2.0 Flash on the free tier** for the LLM — fast, capable, and cost-free.

We also handle edge cases carefully: malformed JSON returns HTTP 400, internal errors return a controlled 500 with no leaked secrets or stack traces, and the service stays up across repeated requests."

SECTION 4 — Live Demo & Testing (2:10 – 2:50)

[SCREEN: Open terminal. Show the live deployed endpoint.]

VOICEOVER:

"Let's see it in action. Our service is deployed on Render at `gridwise-fg71.onrender.com`."

[Run this command on screen:]

```
curl https://gridwise-fg71.onrender.com/health
```

"The health endpoint returns `status: ok` — the service is live and ready."

[Run the optimize-energy curl with a sample scenario — use the SAMPLE-01 from the README. Show the JSON response scrolling.]

```
curl -X POST https://gridwise-fg71.onrender.com/optimize-energy \
-H "Content-Type: application/json" \
-d '{
  "scenario_id": "DEMO-01",
  "operator_notes": [
    "Facilities will wash the rooftop solar panels from noon until 2 PM. During clean
    "The sports office moved next months registration deadline."
  ],
  "hours": [
    {"hour": 0, "demand_kwh": 90, "solar_kwh": 0, "tariff_bdt_per_kwh": 6},
    {"hour": 1, "demand_kwh": 85, "solar_kwh": 0, "tariff_bdt_per_kwh": 6},
    {"hour": 2, "demand_kwh": 80, "solar_kwh": 0, "tariff_bdt_per_kwh": 5},
    {"hour": 3, "demand_kwh": 80, "solar_kwh": 0, "tariff_bdt_per_kwh": 5},
    {"hour": 4, "demand_kwh": 85, "solar_kwh": 0, "tariff_bdt_per_kwh": 5},
    {"hour": 5, "demand_kwh": 95, "solar_kwh": 0, "tariff_bdt_per_kwh": 6},
    {"hour": 6, "demand_kwh": 110, "solar_kwh": 5, "tariff_bdt_per_kwh": 8},
    {"hour": 7, "demand_kwh": 130, "solar_kwh": 20, "tariff_bdt_per_kwh": 10},
    {"hour": 8, "demand_kwh": 150, "solar_kwh": 50, "tariff_bdt_per_kwh": 12},
    {"hour": 9, "demand_kwh": 165, "solar_kwh": 90, "tariff_bdt_per_kwh": 14},
    {"hour": 10, "demand_kwh": 175, "solar_kwh": 130, "tariff_bdt_per_kwh": 16},
    {"hour": 11, "demand_kwh": 180, "solar_kwh": 160, "tariff_bdt_per_kwh": 16},
    {"hour": 12, "demand_kwh": 185, "solar_kwh": 180, "tariff_bdt_per_kwh": 15},
    {"hour": 13, "demand_kwh": 180, "solar_kwh": 170, "tariff_bdt_per_kwh": 14},
    {"hour": 14, "demand_kwh": 170, "solar_kwh": 140, "tariff_bdt_per_kwh": 13},
    {"hour": 15, "demand_kwh": 165, "solar_kwh": 90, "tariff_bdt_per_kwh": 14},
    {"hour": 16, "demand_kwh": 170, "solar_kwh": 45, "tariff_bdt_per_kwh": 18},
    {"hour": 17, "demand_kwh": 185, "solar_kwh": 10, "tariff_bdt_per_kwh": 22},
    {"hour": 18, "demand_kwh": 205, "solar_kwh": 0, "tariff_bdt_per_kwh": 28},
    {"hour": 19, "demand_kwh": 215, "solar_kwh": 0, "tariff_bdt_per_kwh": 30},
    {"hour": 20, "demand_kwh": 205, "solar_kwh": 0, "tariff_bdt_per_kwh": 26},
    {"hour": 21, "demand_kwh": 175, "solar_kwh": 0, "tariff_bdt_per_kwh": 18},
    {"hour": 22, "demand_kwh": 135, "solar_kwh": 0, "tariff_bdt_per_kwh": 10},
    {"hour": 23, "demand_kwh": 105, "solar_kwh": 0, "tariff_bdt_per_kwh": 7}
  ],
  "battery": {
    "capacity_kwh": 220,
    "initial_energy_kwh": 110,
    "minimum_energy_kwh": 40,
    "max_charge_kwh_per_hour": 50,
    "max_discharge_kwh_per_hour": 50
  }
}'
```

"Here's the response. Notice the **directive_interpretation** — Note 0 is correctly identified as solar_reduction with hours [12, 13] and factor 0.25 . Note 1 is classified as no_op because it's about a sports deadline — irrelevant to energy. The **hourly_plan** shows 24 hours of optimized scheduling, and the **total_cost_bdt** is minimized."

[Show running test_samples.py:]

```
python test_samples.py --url https://gridwise-fg71.onrender.com
```

"We also pass all 10 public sample cases — verifying schema conformance, directive accuracy, physical constraints, and cost correctness."

SECTION 5 — Closing (2:50 – 3:00)

[SCREEN: Show architecture diagram one more time, or the GitHub repo README]

VOICEOVER:

"To summarize — GridWise combines the **linguistic flexibility** of Gemini 2.0 Flash with the **mathematical rigor** of a PuLP MILP optimizer, connected through **deterministic guardrails** that ensure no hallucinated constraint ever corrupts the schedule. The result is a robust, cost-optimal energy scheduling service.

Thank you."

Production Notes

Item	Detail
Recording format	Screen recording with voiceover (OBS, Loom, or QuickTime)
Resolution	1080p preferred
Max duration	3:00 sharp

Item	Detail
What to show on screen	Architecture diagram → code snippets → live terminal demo
Tip	Practice the voiceover 2-3 times to stay within 3 minutes. Cut filler words.
Don't forget	Show the deployed URL working live; show test_samples.py passing

Screen Flow Summary

Time	Screen Content
0:00–0:35	Problem statement slide / title
0:35–1:30	Architecture pipeline diagram
1:30–2:10	Code files (<code>llm_interpreter.py</code> , <code>guardrails.py</code> , <code>optimizer.py</code>)
2:10–2:50	Terminal: <code>curl /health</code> → <code>curl /optimize-energy</code> → <code>test_samples.py</code>
2:50–3:00	Architecture diagram or repo README (closing)